

ARE 212 SECTION 6:

Figures with `ggplot2`

Fiona Burlig

February 25, 2016

Loose ends

We didn't really have any loose ends from last time, but just two quick things I wanted to mention:

- I meant to mention this last week, but very few people turned in the proof section of Problem Set 1. You weren't required to, but I strongly urge you to keep up with the proofs, since this will help you on your midterm and final.
- I was asked how to compare two data objects with different column names, etc. You can use the following command:

```
all.equal(itemA, itemB, check.attributes = FALSE)
```

Back to it

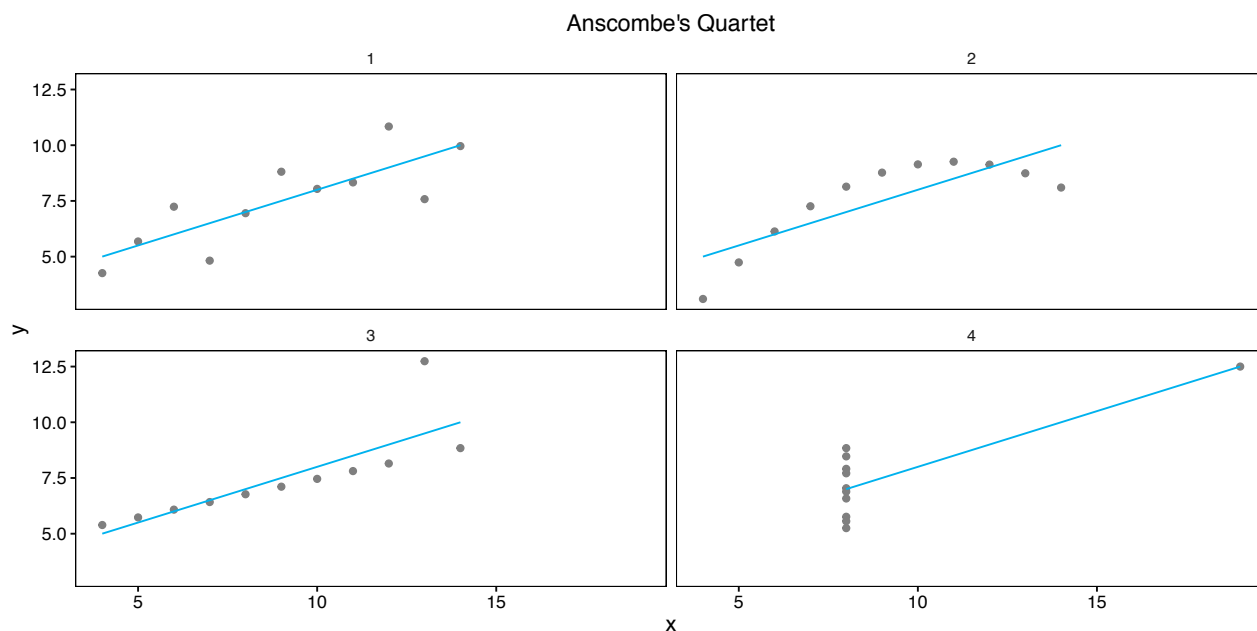
Economists often overlook aesthetics in papers. There are a lot of really ugly and unhelpful images that have made their way into economics journals. A rational response to this might be something like: "Making figures that look nice is time-consuming and challenging, and if the current equilibrium doesn't require it, why should I put in the effort?"

Over the course of this section, I hope to convince you that good graphics are an incredibly important part of doing good research. In most (if not all) cases, you should be able to display the main results of any empirical paper in graphical form.¹ Figures are the easiest way of communicating information to your audience, and if used correctly, can be a very powerful tool in your persuasive arsenal.² A great figure communicates clearly and looks good. Many figures out there do one or neither of these two things - yours should do both. We'll intersperse our discussion of how to make something look nice with the basics of R's extremely powerful and popular graphics package, `ggplot2`.

¹If you can't do this, think carefully about why not, and then fix it.

²A large part of research is, I'm afraid, selling your work.

A first very simple reason for plotting nicely is that it's very important for knowing what your data actually look like! Why do we care about *that*? These four figures all have the same mean x , mean y , variance of x and y , correlation between y and x , and linear best-fit line - but obviously have very different properties!³



Yeah.

We'll get started by installing the `ggplot2` package:

```
install.packages('ggplot2')
```

As usual, we'll load a variety of packages, grab a couple of functions, and bring our favorite dataset back:

```
##### PACKAGES #####
library(dplyr)
library(magrittr)
library(ggplot2)

##### FUNCTIONS #####
tbl_dfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
    as.matrix()
}

# New function: turns data into tbl_df's in one step!
as_tbl_df <- function(data) {
  dataset <- as.data.frame(data) %>%
    tbl_df()
}
```

³These datasets are called Anscombe's Quartet, and you can find more about them [here](#).

```

smallOLS <- function(data, y, X) {
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  return(betahat)
}

OLS <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  # warn the user
  if (identical(xdata[,1], rep(1, n)) == FALSE) {
    olsWarn <- "NO INTERCEPT DETECTED"
    stop(olsWarn)
  }
  # beta
  betahat <- smallOLS(data, y, X)
  # resids
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
  #std error
  se <- sqrt(s2 * diag(XpXinv))
  # t statistic & its pvalue
  tStat <- (betahat - 0) / se
  pVal <- tStat %>%
    abs() %>%
    "*"(-1) %>%
    pt(df = n - k) %>%
    "*" (2)
  #Calculate F stat & its pvalue
  J <- length(X) - 1
  bigR <- diag(J) %>%
    cbind(0, .)
  RXXinvR <- solve(bigR %*% solve(t(xdata) %*% xdata) %*% t(bigR))
  FStat <- t(bigR %*% betahat) %*% RXXinvR %*% (bigR %*% betahat) / (s2 * J)
  pValF <- 1 - pf(FStat, df1 = J, df2 = n-k)

  olsOut <- list(betahat, se, tStat, pVal, FStat, pValF)
  names(olsOut) <- c("betahat", "SE", "tStat", "tStatPVal", "FStat", "FStatPVal")
  return(olsOut)
}

##### DATA #####

```

```
autos <- read.csv("autos.csv") %>%
  as.tbl_df()

# add our column of ones:
autos <- mutate(autos, ones = 1)
```

Before we go too much further, I want to introduce you to a new data type in R, the factor. Factors are how R keeps track of categorical variables. We're going to tell R to treat `foreign` as a categorical variable, with labels "domestic" and "foreign":

```
autos <- mutate(autos, foreign = factor(foreign, labels = c("domestic", "foreign")))
```

Okay, moving on...

Scatterbrained

`ggplot2` operates according to the "grammar of graphics."⁴ The idea is simple, but very powerful: just like we build sentences from different elements, so too can we build figures. If you're used to making figures in other statistical packages, things are going to feel unwieldy for a little bit here - but bear with me. Becoming literate in `ggplot2` means that you'll be able to create highly-customizable figures - it'll be a little bit trickier than base R graphics commands like `plot()` in the short run, but you'll thank me later.

Graphics in `ggplot2` are built out of component parts, called *layers*. The first thing we need to do to construct a plot object in R is to call the `ggplot()` function, and assign its output to a new object. We'll practice by creating a new plot object, `myPlot`. The beginning of your function call will be:

```
myPlot <- ggplot()
```

Notice that just like every other time we assign an object to a name, R won't actually display the object without us asking nicely. So let's do that:

```
myPlot
```

Oops - we got an error. `ggplot()` doesn't know how to handle a blank plot. So let's add some layers! Let's start by creating a scatterplot layer, in which we plot MPG against weight.⁵ We start with a *geometry*: this is what's actually going to be plotted. We'd like our graph to be a regular scatter plot with dots, so we'll use the function `geom_point()`. We'll also specify a dataset, as well as an *aesthetic mapping*. This tells R how to use different objects in our dataset.

```
scatterLayer <- geom_point(data = autos,
  mapping = aes(x = weight, y = mpg))
```

Just looking at the `scatterLayer` object isn't very exciting. Let's add it to our plot. We do this "adding" with the `+` sign.⁶

⁴The original citation for this can be found [here](#).

⁵And you thought that maybe, just maybe, we'd finally escaped car data. Not today, my friends, not today.

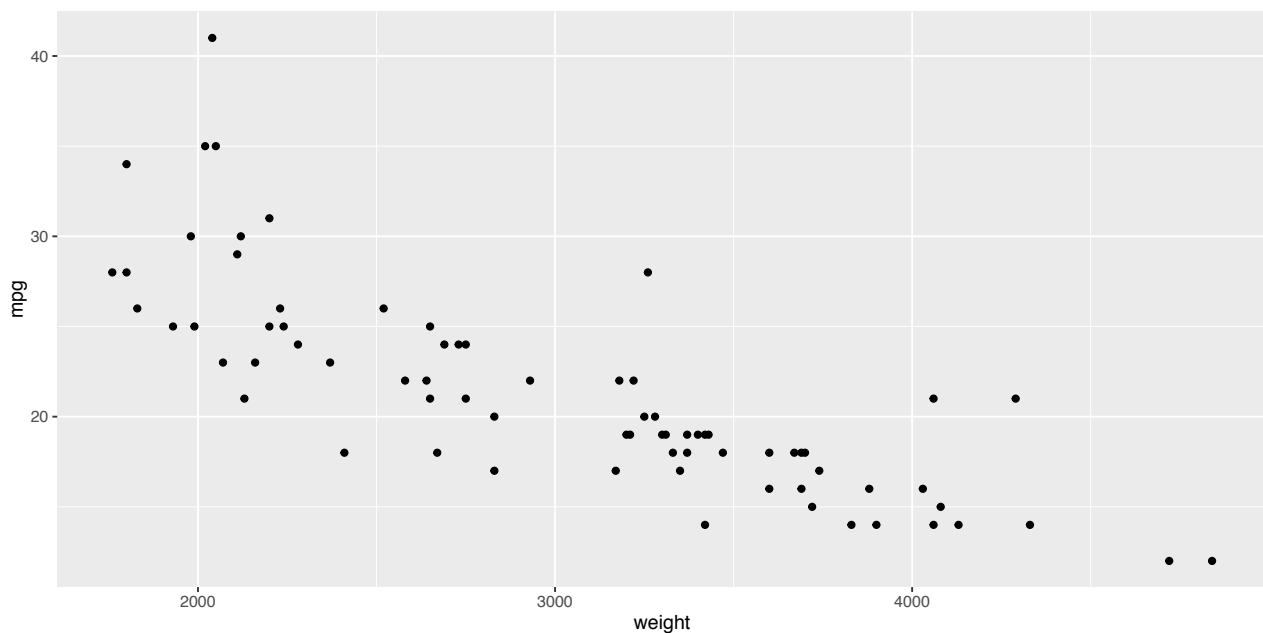
⁶I appreciate Hadley Wickham's cheeky sense of humor.

```
scatterLayer

## mapping: x = weight, y = mpg
## geom_point: na.rm = FALSE
## stat_identity: na.rm = FALSE
## position_identity
```

```
myPlot <- ggplot() + scatterLayer
```

```
myPlot
```



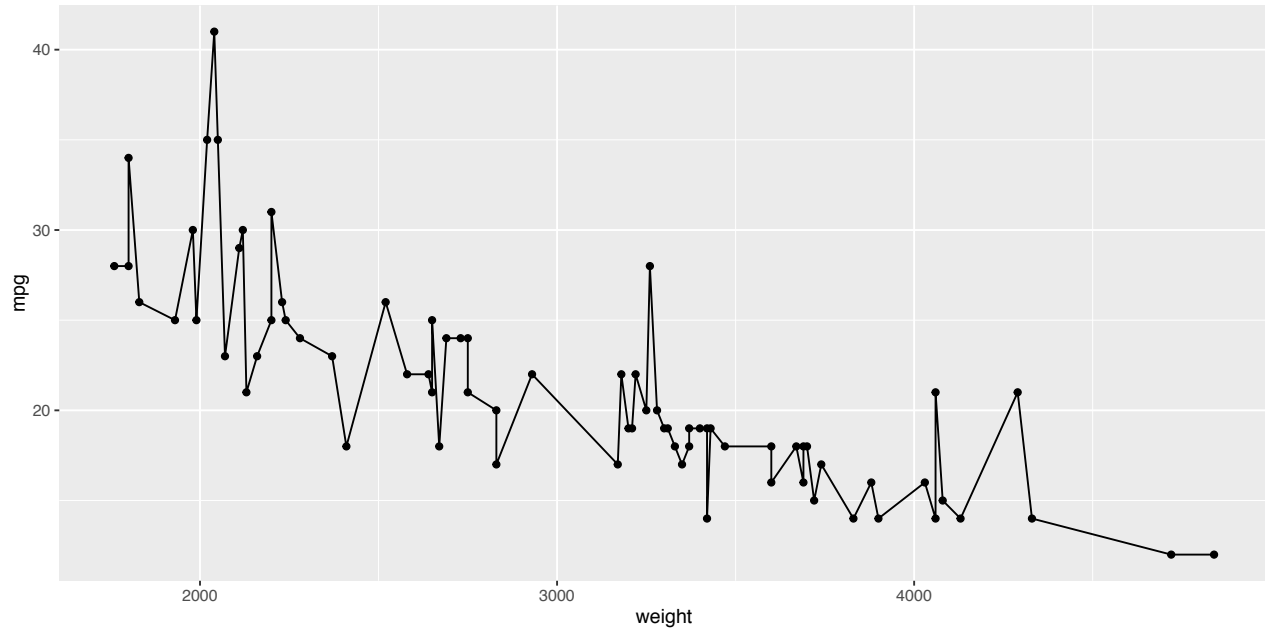
Awesome! Of course, I told you that `ggplot2` makes super customizable graphics. Let's delve more deeply into this by adding a best-fit line through our points. We'll do that with (you guessed it) another layer:

```
lineLayer <- geom_line(data = autos,
  mapping = aes(x = weight, y = mpg))
```

And now combine:

```
myPlot <- ggplot() + scatterLayer + lineLayer
```

```
myPlot
```



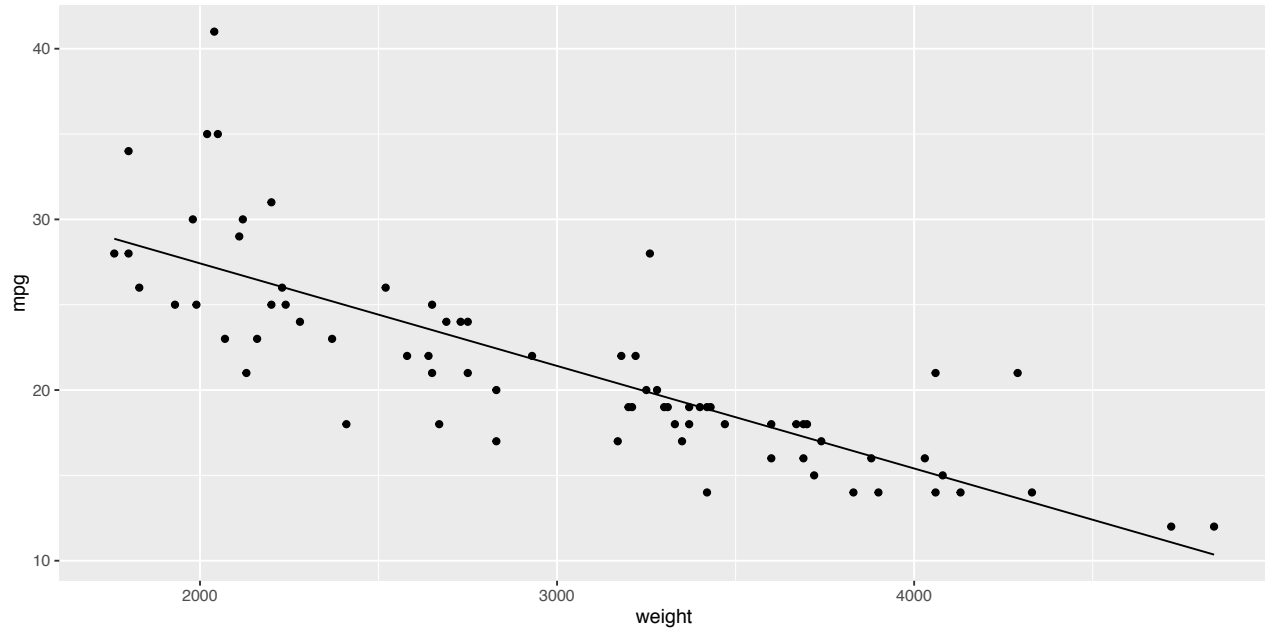
Oops. That wasn't quite what I wanted. I told you I wanted a *fitted* line. We can do this in two ways, so I'll show you both. The first is to actually calculate fitted $\hat{y}$ values, so we'll start there:

```
betaHats <- OLS(autos, "mpg", c("ones", "weight")) %>%  
  "$"(betahat)  
autos <- mutate(autos, yhat = betaHats[1] + betaHats[2] * weight)
```

And now we can actually put our fitted line onto our plot:

```
fittedLineLayer <- geom_line(data = autos,  
  mapping = aes(x = weight, y = yhat))  
myPlot <- ggplot() + scatterLayer + fittedLineLayer
```

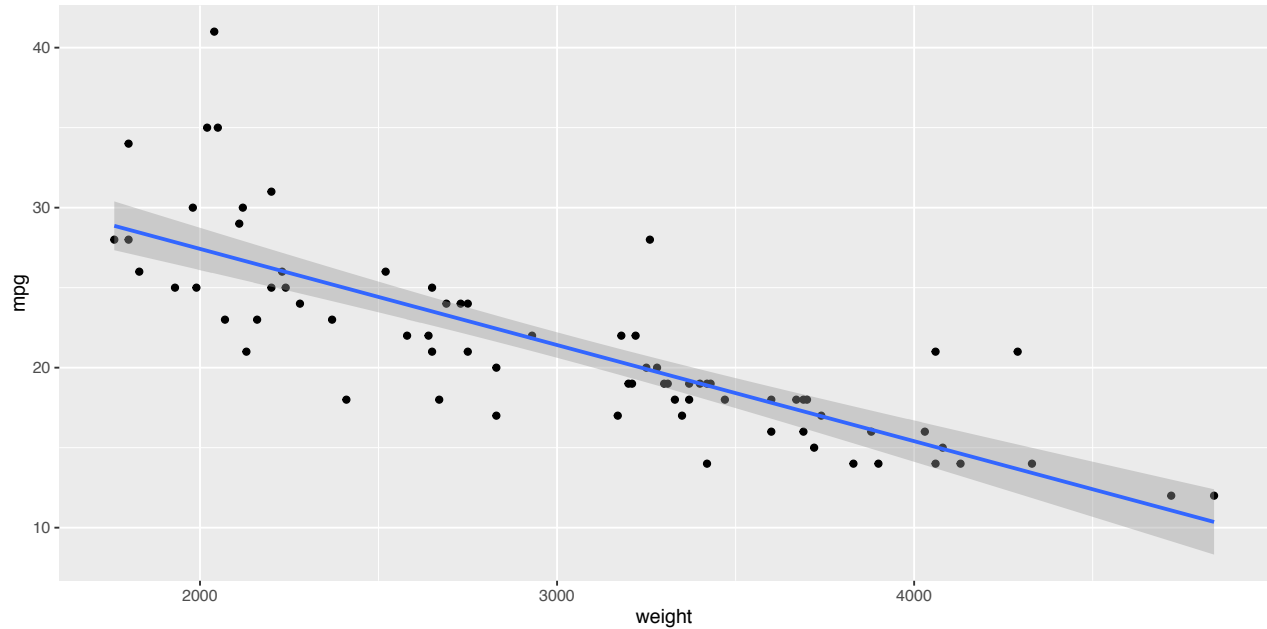
myPlot



The other way to do this is to use the `geom_smooth()` geometry:

```
smoothLineLayer <- geom_smooth(data = autos,  
  mapping = aes(x = weight, y = mpg),  
  # calculate the fitted line using LM  
  method = "lm")  
  
myPlot <- ggplot() + scatterLayer + smoothLineLayer
```

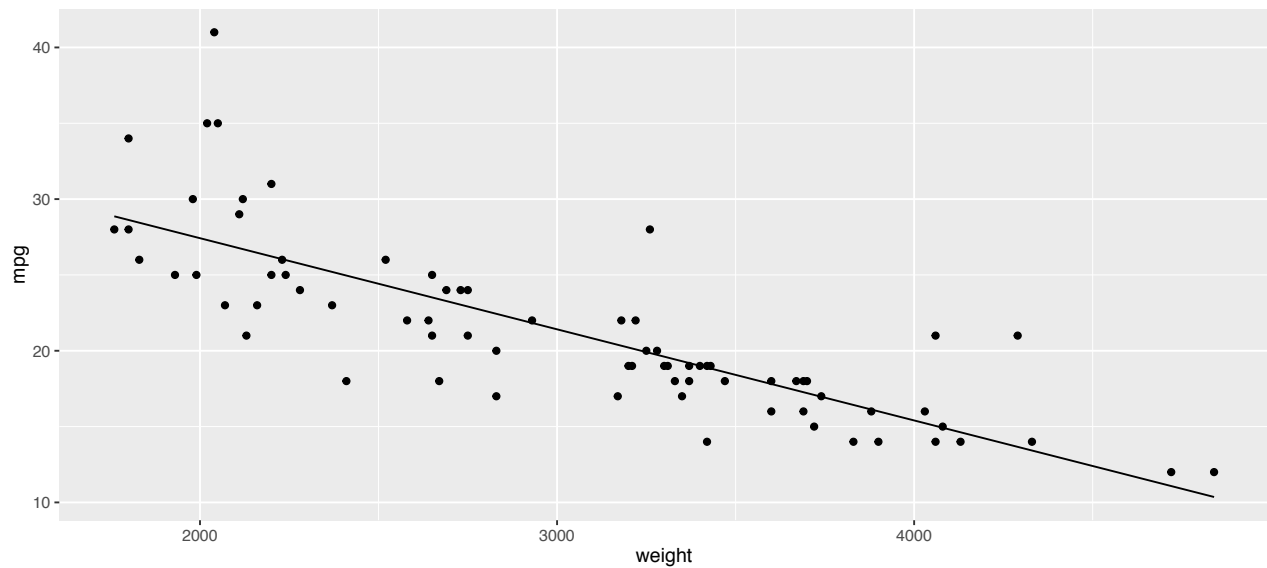
```
myPlot
```



So, that's nice - but it actually uses a canned routine (`lm()`) and is therefore banned from the course. For fear of incurring Max's wrath, let's go back to our own fitted line:

```
myPlot <- ggplot() + scatterLayer + fittedLineLayer
```

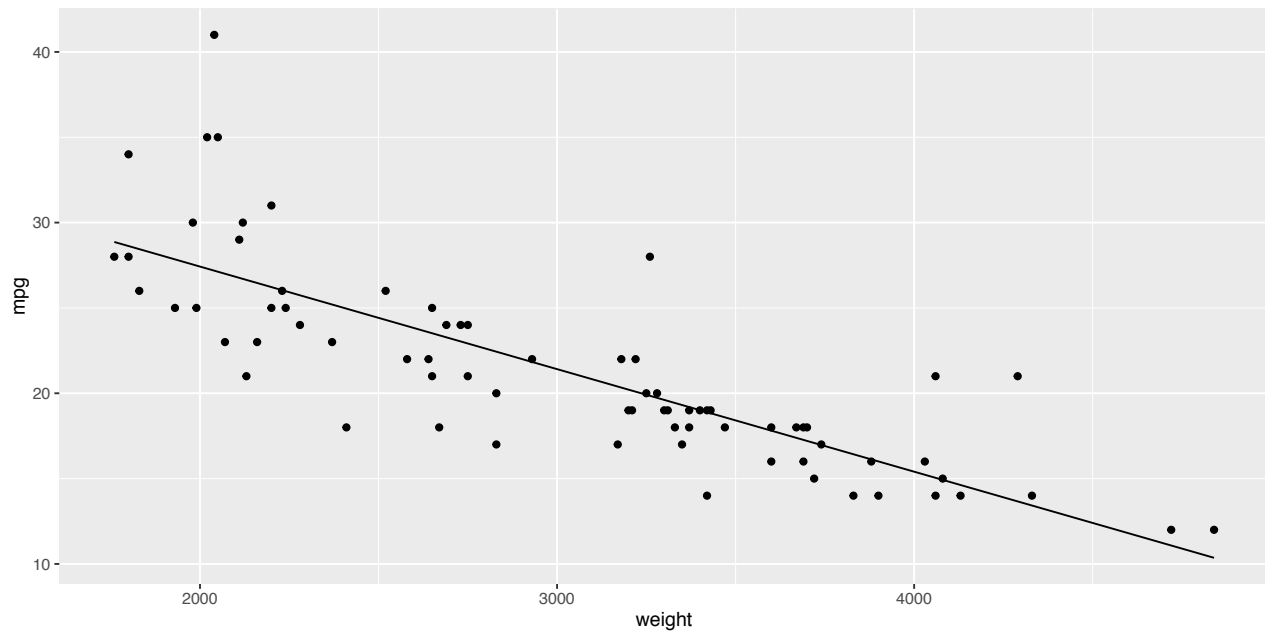
```
myPlot
```



Notice that we can produce the same plot with the following code:

```
myPlot <- ggplot() +
  # this was our scatter layer
  geom_point(data = autos, mapping = aes(x = weight, y = mpg)) +
  # this was our line layer
  geom_line(data = autos, mapping = aes(x = weight, y = yhat))
```

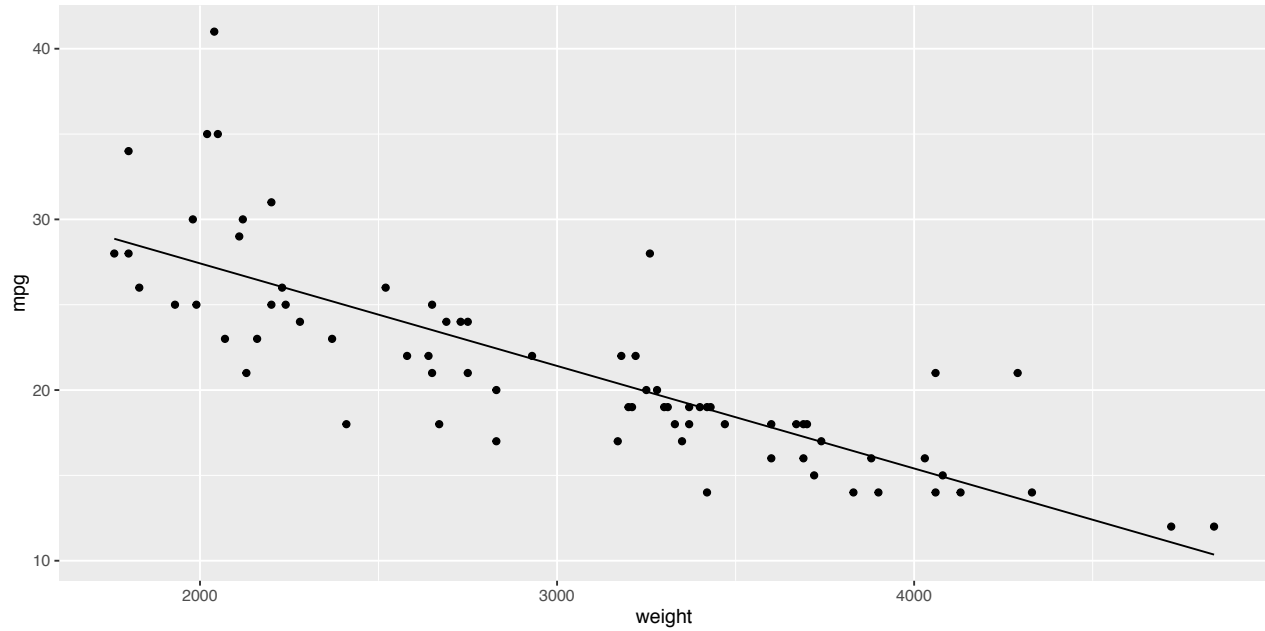
myPlot



But that's kind of redundant and annoying. Notice that both of these objects share two things in common: the dataset and the x variable. `ggplot2`'s creator, Hadley Wickham, is all about efficiency in code - so there must be a better way to do this. Turns out that there is. Rather than specifying data and aesthetics for each layer, we can *also* set a base dataset and aesthetic, like this:

```
myPlot <- ggplot(data = autos, aes(x = weight)) +
  geom_point(aes(y = mpg)) +
  geom_line(aes(y = yhat))
```

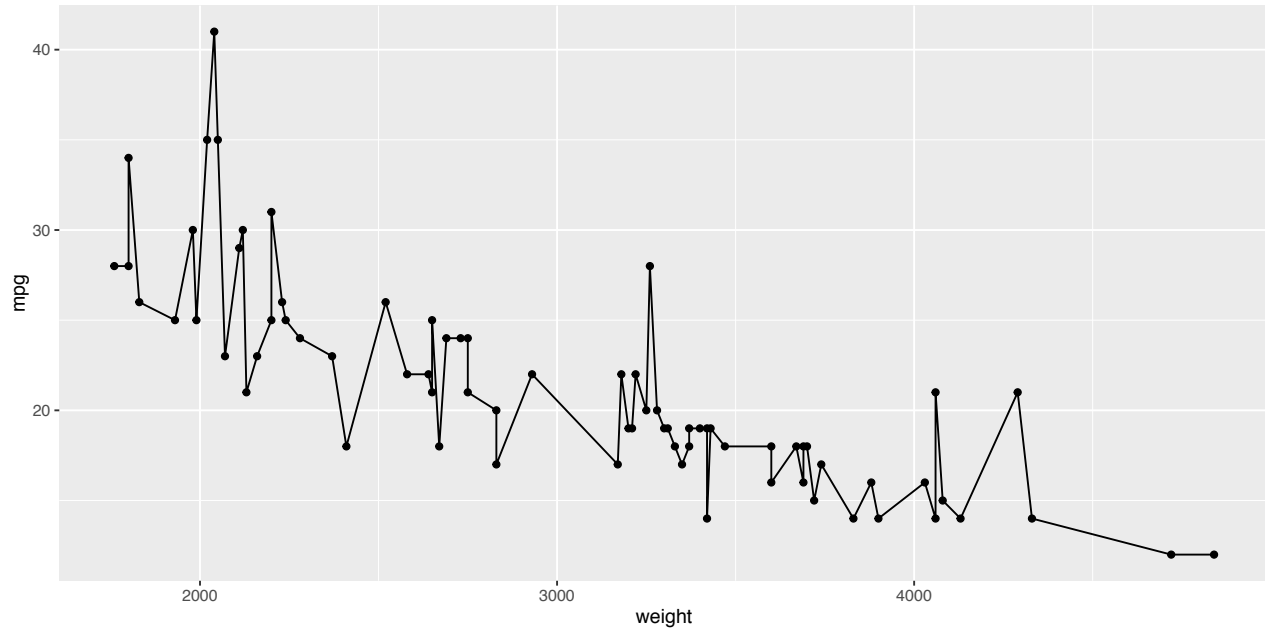
myPlot



Much better. If both lines had shared a y variable, we could have simplified this down even further. Consider the (accidental) connected line plot we made before:

```
myConnectedPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +  
  geom_point() +  
  geom_line()
```

```
myConnectedPlot
```



Aaaaah, perfection.

How to Win Friends and Influence People

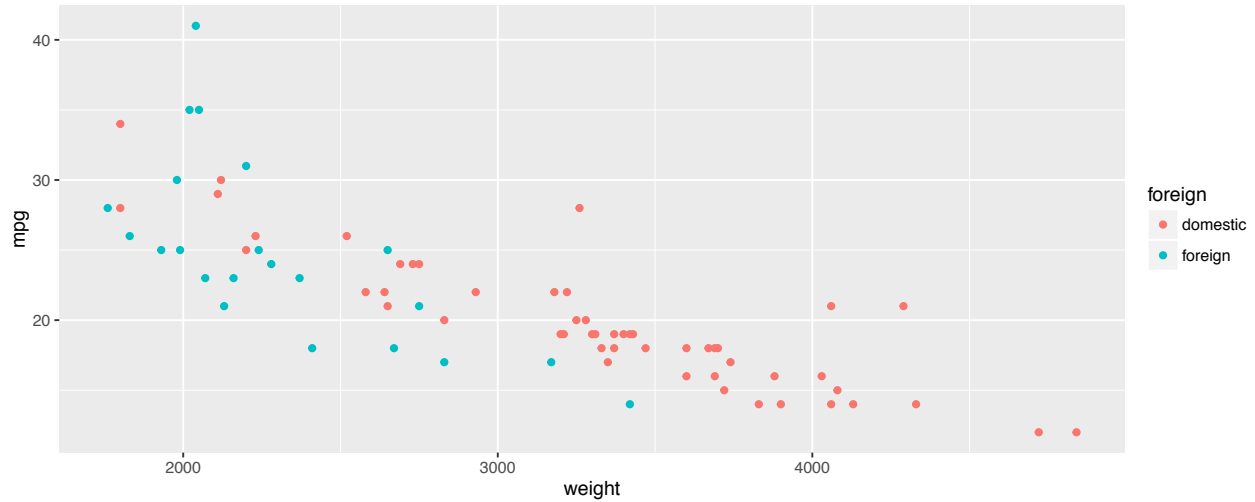
What's the secret? Pretty figures.

Another thing we might be interested in doing is highlighting which observations in our dataset are domestic cars and which are foreign. We can do this by editing our aesthetics slightly.⁷

```
myFactoredPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +  
  geom_point(aes(color = foreign))
```

⁷This is for you, person who asked me 30' ago why we'd be using factor variables.

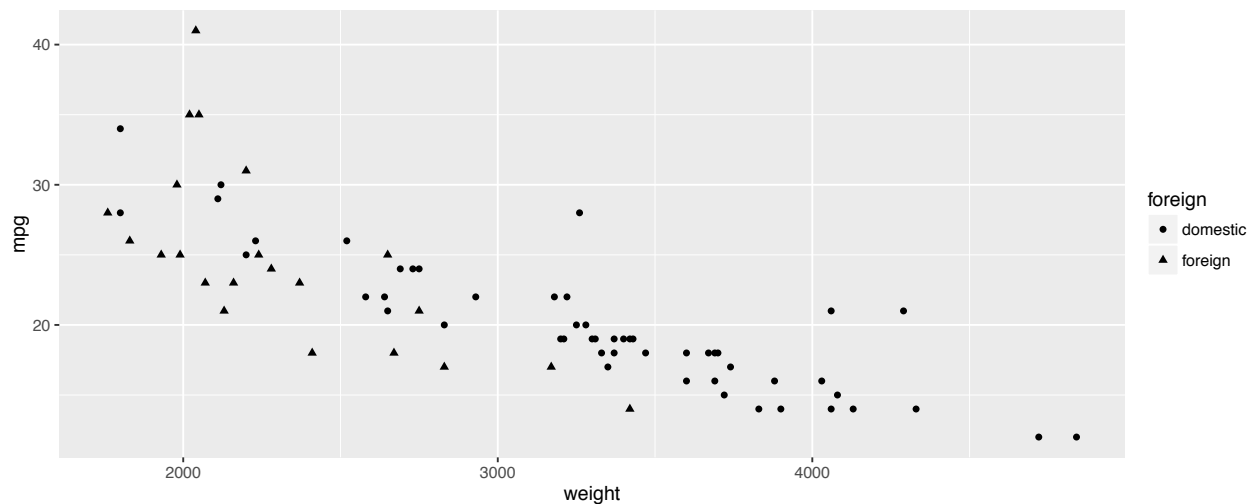
myFactoredPlot



Notice that R now gives us different colors, which allows us to differentiate between foreign and domestic cars easily. This distinction will go away when we print things in black and white, though - and there are a bunch of color-blind people out there - so I strongly prefer marker shapes to colors for distinguishing between observation types. Thankfully, R can easily accomodate that too:

```
myFactoredPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +  
  geom_point(aes(shape = foreign))
```

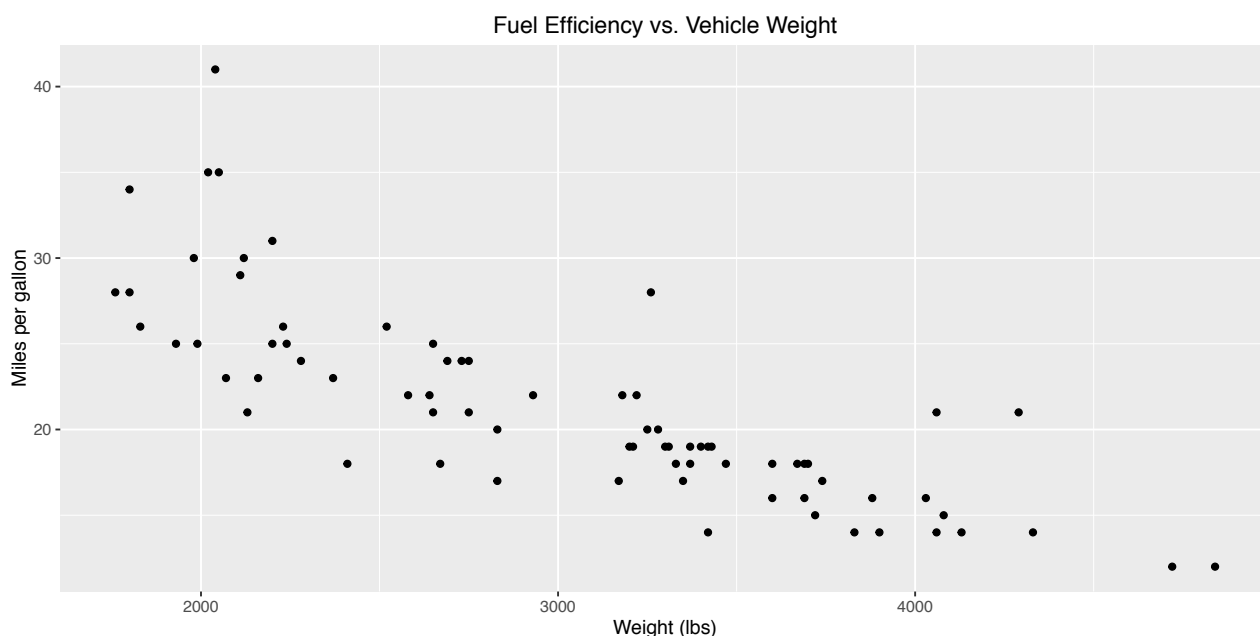
myFactoredPlot



Okay, cool. Now we have an idea of how to make figures using `ggplot()`, but we haven't spent any time talking about how to make those figures actually look good. The first thing we need to do is to add a title to our figure, and to make our axis titles more descriptive. Let's go back to the base plot for this. Just like we added geometric layers, we can also add axis labels:

```
myLabeledPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +
  geom_point() +
  # graph title
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +
  # x & y axis labels
  xlab("Weight (lbs)") + ylab("Miles per gallon")
```

myLabeledPlot



Edward Tufte has made his career out of studying the art of displaying data.⁸ One of his central tenets is not to “waste ink” - that is, don't have any extraneous color or shading on a page. Strip a figure down to its minimally necessary elements. I think this is too often overlooked. One of my own personal central graphical tenets? Don't make it obvious what software you used to make your figures.⁹ In order to avoid detection, we'll have to do some customization. `ggplot()` has a really easy method for tweaking figures such that they're exactly to our liking: `theme()`. We can adjust pretty much any element of a `ggplot()` figure using the `theme()` call. In particular, I think the standard gray `ggplot()` background is an ugly waste of ink. Let's get rid of it using `theme()`:

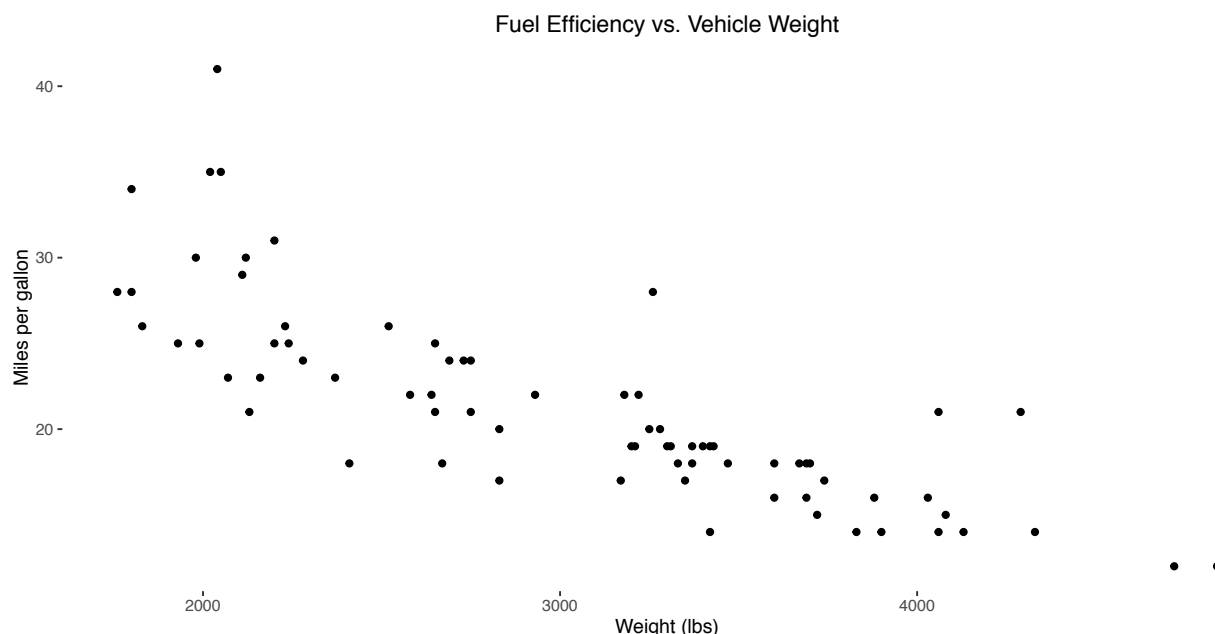
```
myGreatPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +
  geom_point() +
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +
  xlab("Weight (lbs)") + ylab("Miles per gallon") +
```

⁸Check his stuff out [here](#).

⁹Here's looking at you, hideous blue Stata background!

```
# set the background
theme(panel.background = element_rect(fill = NA))
```

myGreatPlot



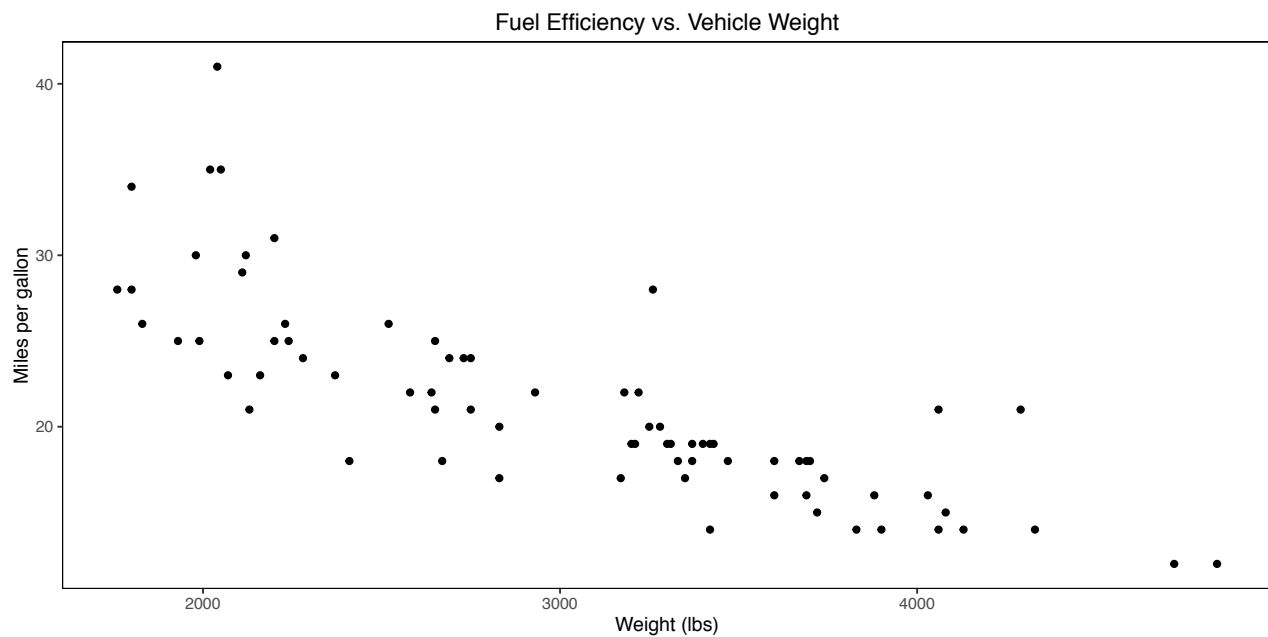
That's a little better - but we've lost too much. Let's bring the border around our plot back:

```
myGreatPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +
  geom_point() +
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +
  xlab("Weight (lbs)") + ylab("Miles per gallon") +
  # set the background
  theme(panel.background = element_rect(fill = NA),
        # draw the border
        panel.border = element_rect(fill = NA, color = "black"))
```

I want to make a few more tweaks before I'm totally satisfied. In particular, these axis labels being gray and small rather than black and a legible size annoys me:

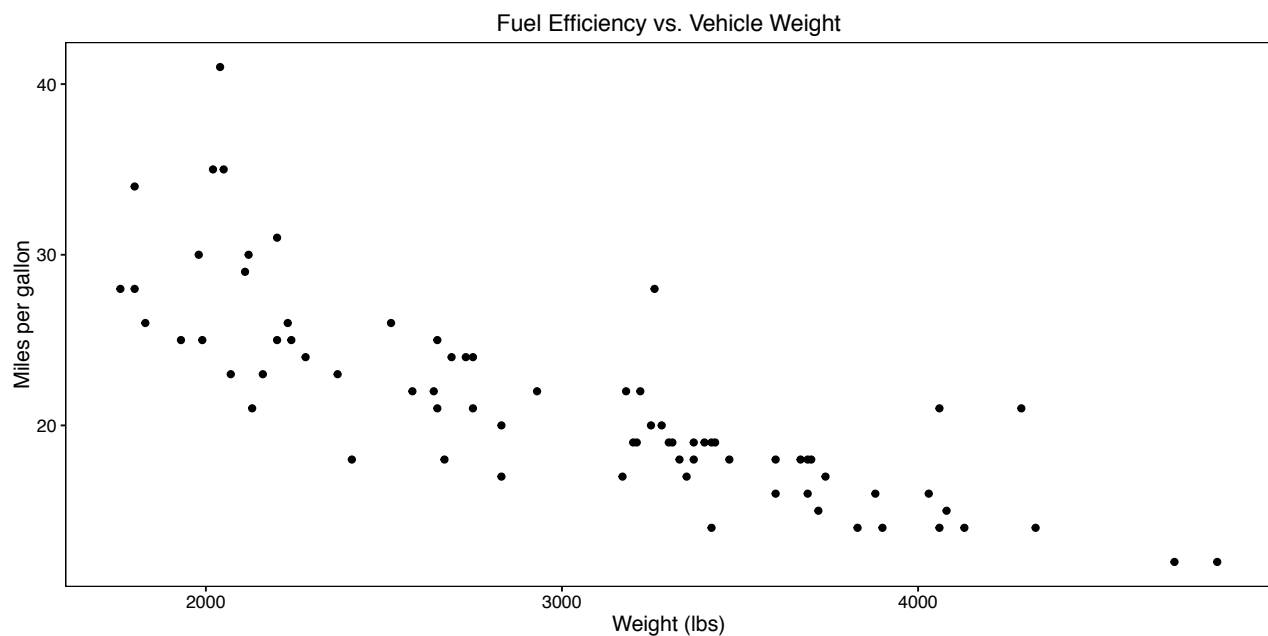
```
myGreatPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +
  geom_point() +
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +
  xlab("Weight (lbs)") + ylab("Miles per gallon") +
  # set the background
  theme(panel.background = element_rect(fill = NA),
        # draw the border
        panel.border = element_rect(fill = NA, color = "black"),
        # make the axis ticks black
```

myGreatPlot



```
axis.ticks = element_line(color = "gray5"),  
# make the axis tick text black, and fontsize 10  
axis.text = element_text(color = "black", size = 10),  
# make the axis label black and size 12  
axis.title = element_text(color = "black", size = 12))
```

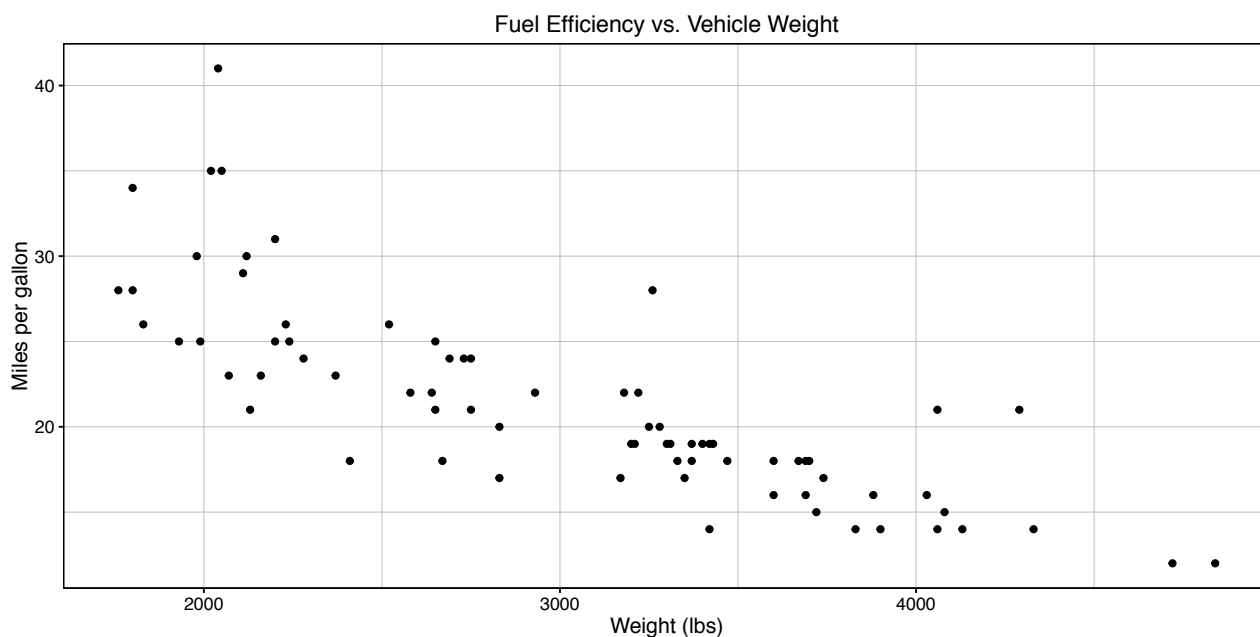
myGreatPlot



Not having grid lines might bother you. Actually, it's not that we don't currently have grid lines - but they're defaulted to white, so they're not showing up on our white (clear) background. I don't think this plot needs them, but if you really want, you can add them back in like this:

```
myGreatPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +  
  geom_point() +  
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +  
  xlab("Weight (lbs)") + ylab("Miles per gallon") +  
  theme(panel.background = element_rect(fill = NA),  
        panel.border = element_rect(fill = NA, color = "black"),  
        panel.grid.major = element_line(color = "gray", size = 0.1),  
        panel.grid.minor = element_line(color = "gray", size = 0.1),  
        axis.ticks = element_line(color = "gray5"),  
        axis.text = element_text(color = "black", size = 10),  
        axis.title = element_text(color = "black", size = 12))
```

myGreatPlot



Again, I find having both x and y gridlines to be visually unnecessary. Let's keep only the "major" horizontal ones:

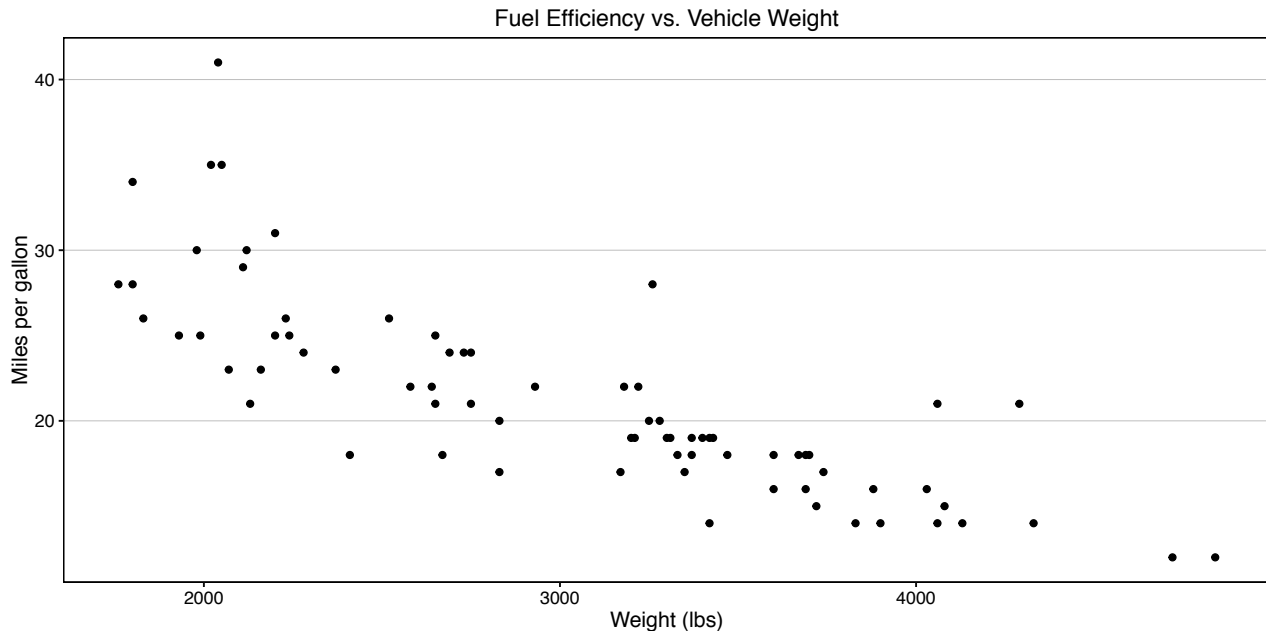
```
myGreatPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +  
  geom_point() +  
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +  
  xlab("Weight (lbs)") + ylab("Miles per gallon") +  
  theme(panel.background = element_rect(fill = NA),  
        panel.border = element_rect(fill = NA, color = "black"),  
        panel.grid.major.y = element_line(color = "gray", size = 0.1),
```

```

panel.grid.major.x = element_blank(),
panel.grid.minor.y = element_blank(),
panel.grid.minor.x = element_blank(),
axis.ticks = element_line(color = "gray5"),
axis.text = element_text(color = "black", size = 10),
axis.title = element_text(color = "black", size = 12))

```

myGreatPlot



Much better. If we want to use the same `theme()` code each time we make a figure, we can also assign it to an object to streamline our `ggplot()` experience (I'm going to turn my grid lines back off, too):

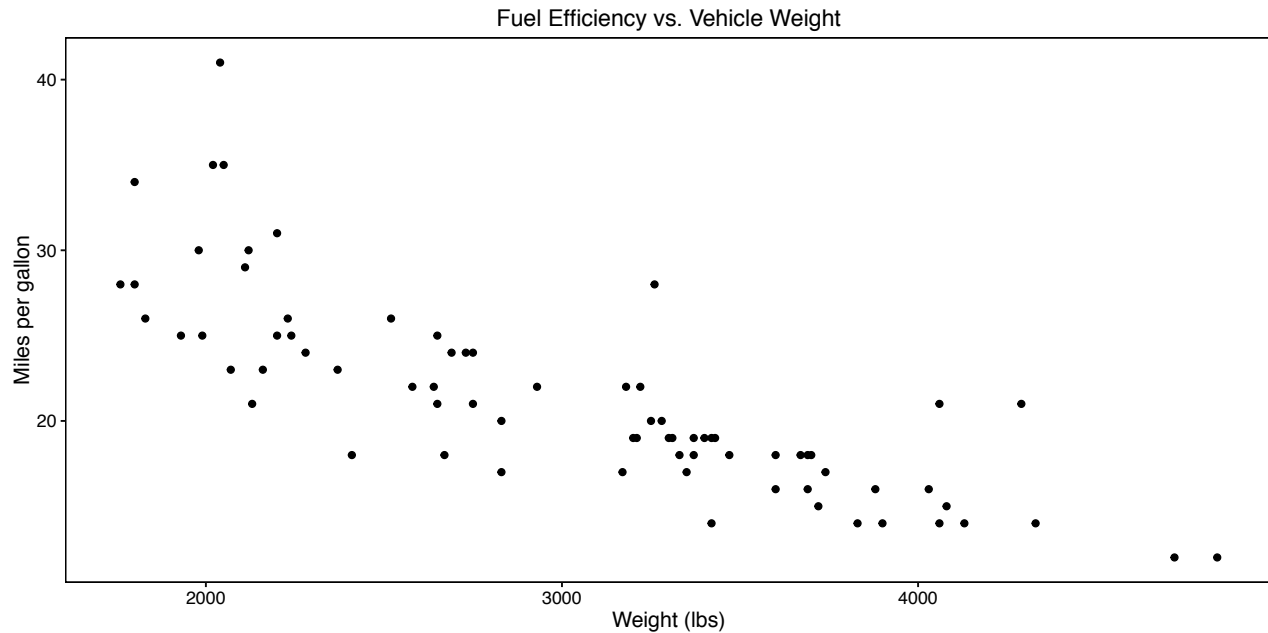
```

myThemeStuff <- theme(panel.background = element_rect(fill = NA),
  panel.border = element_rect(fill = NA, color = "black"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  axis.ticks = element_line(color = "gray5"),
  axis.text = element_text(color = "black", size = 10),
  axis.title = element_text(color = "black", size = 12))

myGreatPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +
  geom_point() +
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +
  xlab("Weight (lbs)") + ylab("Miles per gallon") + myThemeStuff

```

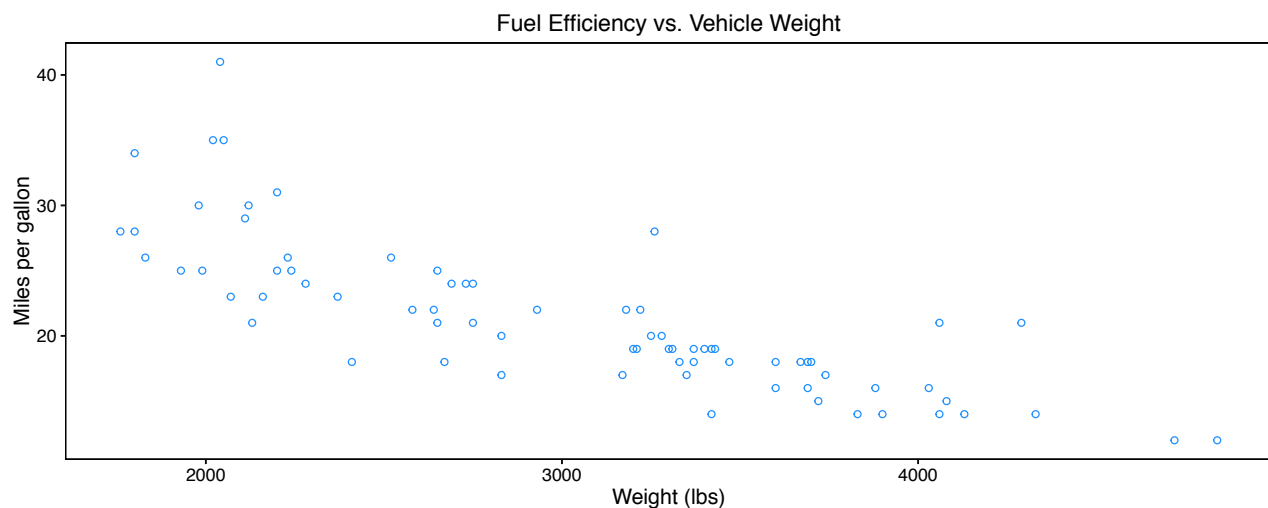
myGreatPlot



That's much nicer code. "But, Fiona!" you gripe, "Those black dots don't look amazing. What if I wanted a different color? Or a different shape?" You should've guessed that we can handle that too.¹⁰

```
myColoredPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +  
  geom_point(color = "dodgerblue", shape = 21) +  
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +  
  xlab("Weight (lbs)") + ylab("Miles per gallon") + myThemeStuff
```

myColoredPlot

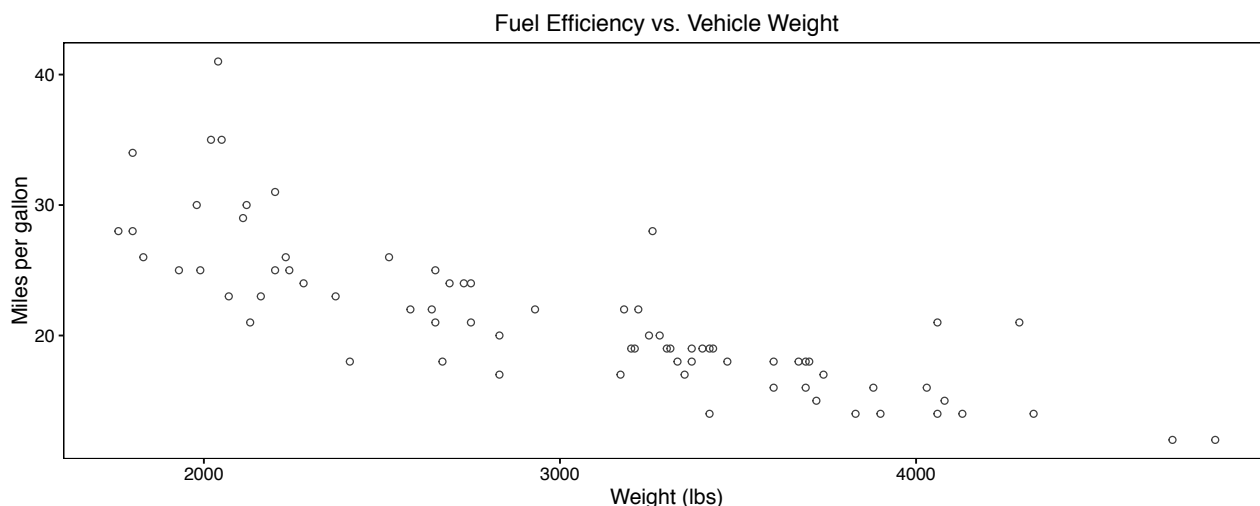


¹⁰There are a ton of available color options in R. See [here](#) for a handy PDF with a bunch.

I like that color - but I'm morally opposed to the name, so we have to change it.¹¹ How about this? (Have I mentioned that I really like grayscale?)

```
myColoredPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +  
  geom_point(color = "gray25", shape = 21) +  
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +  
  xlab("Weight (lbs)") + ylab("Miles per gallon") + myThemeStuff
```

myColoredPlot

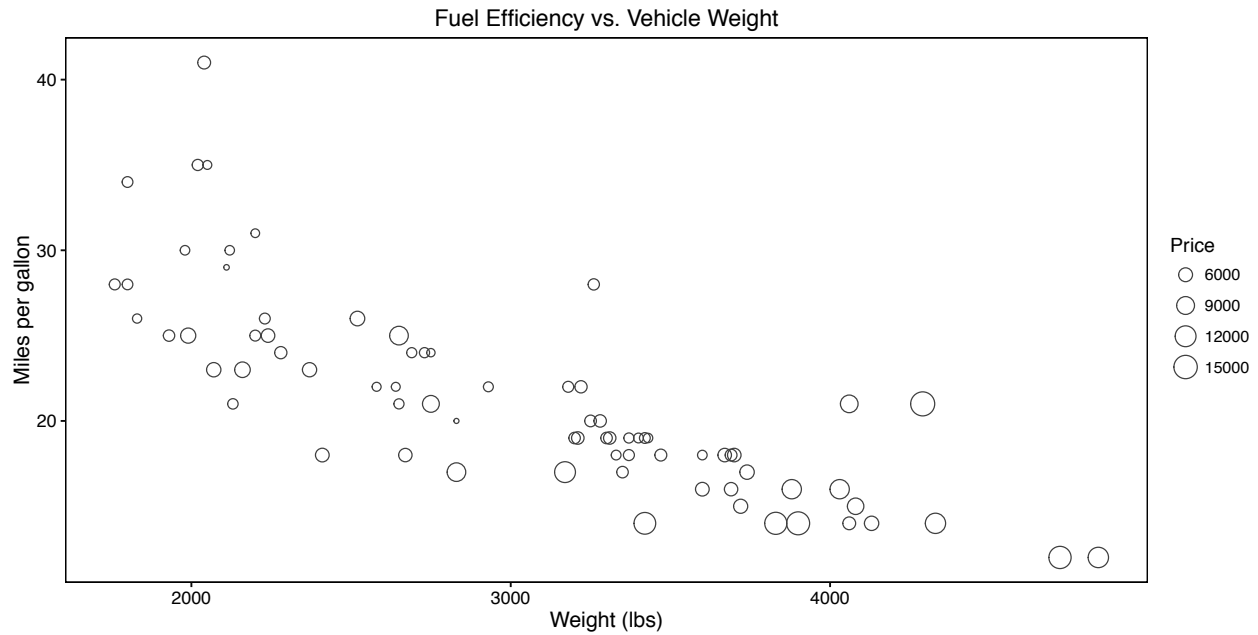


Cool. One last thing we can do with the scatterplot, that you'll occasionally see, is marker sizes that indicate something. We'll make our markers scale with price, by setting the size. Don't forget to adjust the legend with the `labs()` command. We'll also update `myThemeStuff` to remove the annoying gray background from the legend too:

```
myThemeStuff <- theme(panel.background = element_rect(fill = NA),  
  panel.border = element_rect(fill = NA, color = "black"),  
  panel.grid.major = element_blank(),  
  panel.grid.minor = element_blank(),  
  axis.ticks = element_line(color = "gray5"),  
  axis.text = element_text(color = "black", size = 10),  
  axis.title = element_text(color = "black", size = 12),  
  legend.key = element_blank())  
  
myColoredPlot <- ggplot(data = autos, aes(x = weight, y = mpg, size = price)) +  
  geom_point(color = "gray25", shape = 21) +  
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +  
  xlab("Weight (lbs)") + ylab("Miles per gallon") + labs(size = "Price") +  
  myThemeStuff
```

¹¹Go Giants! Whooooo even year whoooo!

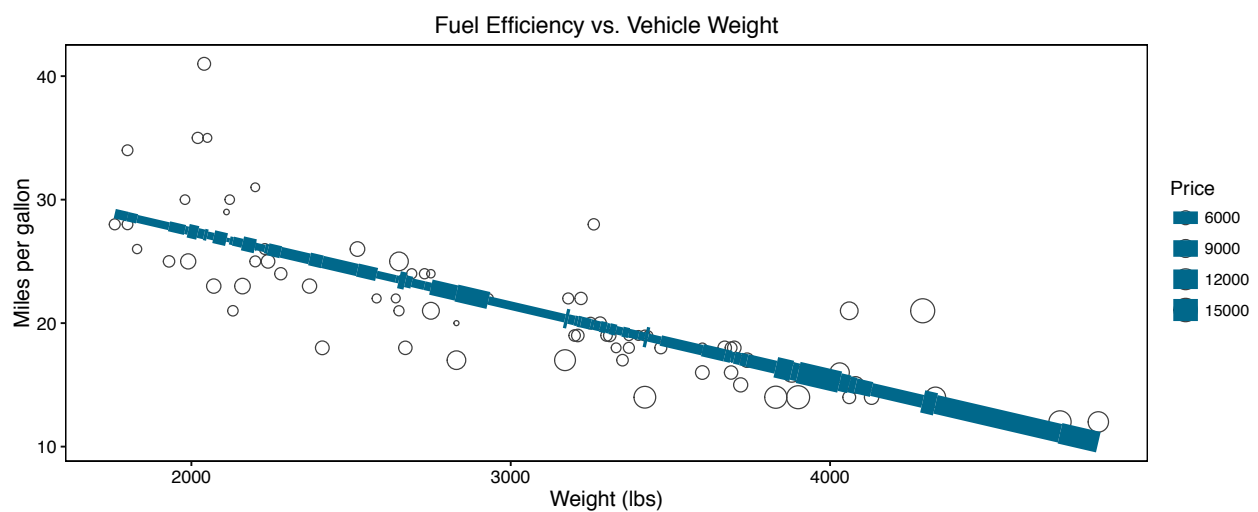
myColoredPlot



Let's finish our scatterplot by adding our fit line back in:

```
myColoredPlot <- ggplot(data = autos, aes(x = weight, y = mpg, size = price)) +  
  geom_point(color = "gray25", shape = 21) +  
  geom_line(aes(y = yhat), color = "deepskyblue4") +  
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +  
  xlab("Weight (lbs)") + ylab("Miles per gallon") + labs(size = "Price") +  
  myThemeStuff
```

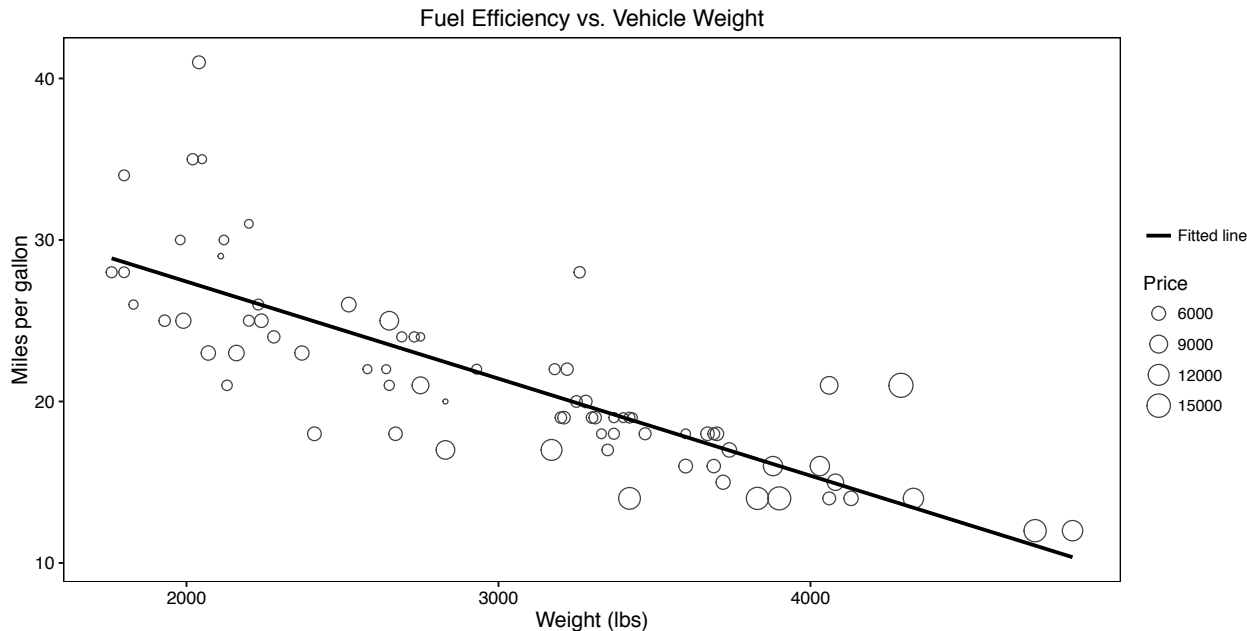
myColoredPlot



Ew! What happened? We specified that size should be determined by price in our base options, rather than just for the points. Let's edit:

```
myColoredPlot <- ggplot(data = autos, aes(x = weight, y = mpg)) +  
  geom_point(aes(size = price), color = "gray25", shape = 21) +  
  # add the line. color = "Fitted line" names our fitted line  
  # which will let us link it to a color value below  
  geom_line(aes(y = yhat, color = "Fitted line"), size = 1) +  
  # tells R to set color our fitted line black  
  scale_color_manual(values = c("Fitted line" = "black")) +  
  ggtitle("Fuel Efficiency vs. Vehicle Weight") +  
  xlab("Weight (lbs)") + ylab("Miles per gallon") + labs(color = "", size = "Price") +  
  myThemeStuff
```

myColoredPlot



That's better.

geoms for days

We just spent a *loooooong* time on a scatter plot. Making high-quality figures takes a long time - just like running high-quality regressions. But scatter plots are only one way we might want to display data. Let's delve into a couple of other `geom` types. We'll start with that old workhorse, the histogram:

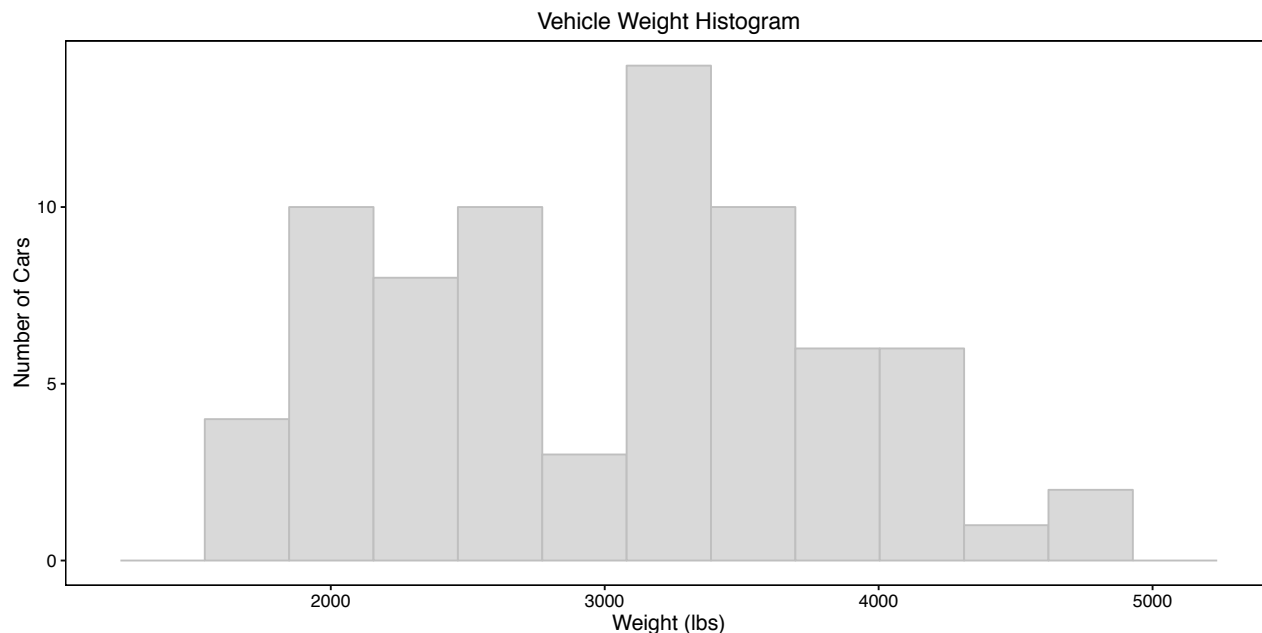
```
#plotting weight only  
myHistogram <- ggplot(data = autos, aes(x = weight)) +  
  # set the number of bins  
  geom_histogram(bins = 10,
```

```

    # and the color/fill
    color = "gray75", fill = "gray85") +
  ggtitle("Vehicle Weight Histogram") +
  xlab("Weight (lbs)") + ylab("Number of Cars") + myThemeStuff

```

```
myHistogram
```



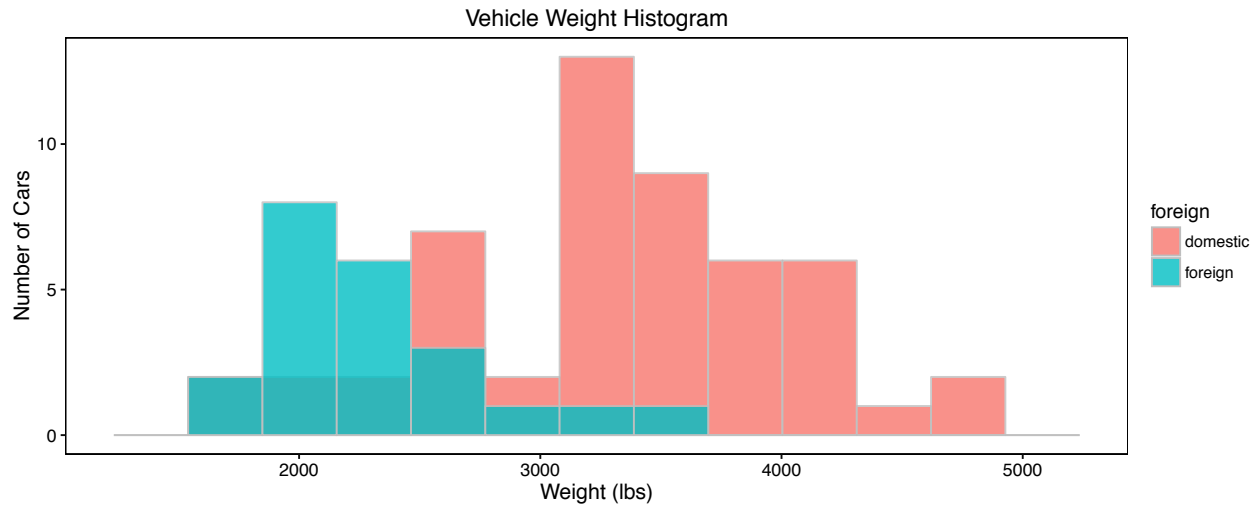
What I really want to see, though, is the histogram of weights for domestic cars and for foreign cars. We'll use the `alpha` setting to make our histograms somewhat transparent, so that we can layer them nicely. Note that when you use multiple overlaid histograms with `ggplot2`, you should always set the `position` to `'identity'`, which tells R to colocate histograms rather than stacking them:

```

myForeignHistogram <- ggplot(autos, aes(x = weight, fill = foreign)) +
  # alpha = 0.8 sets the opacity of the histograms
  geom_histogram(bins = 10, alpha = 0.8, position = "identity", color = "gray") +
  ggtitle("Vehicle Weight Histogram") +
  xlab("Weight (lbs)") + ylab("Number of Cars") + myThemeStuff

```

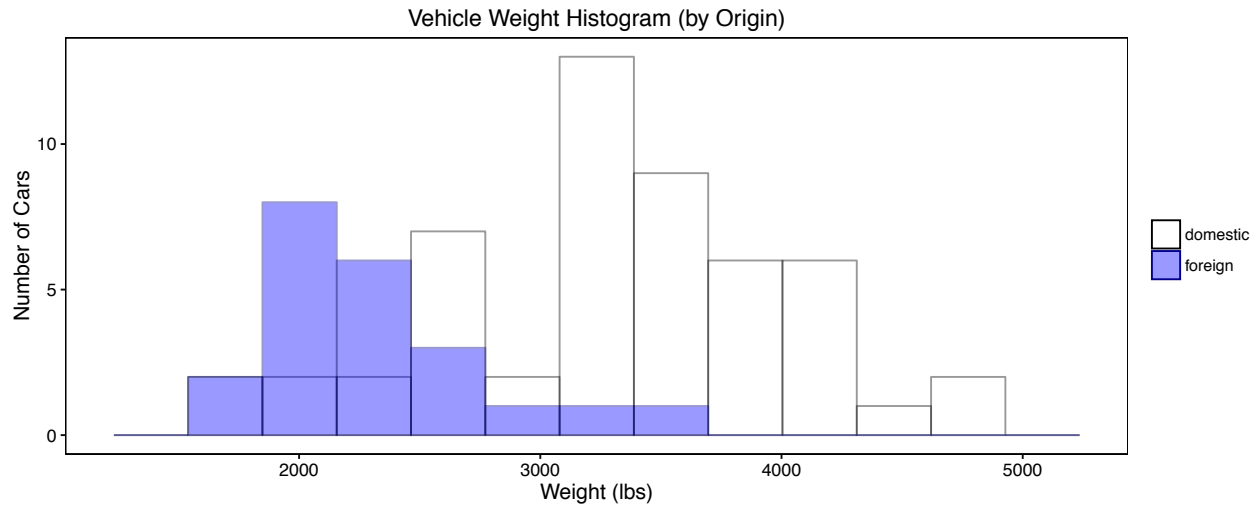
```
myForeignHistogram
```



Okay, that's kind of what we wanted - but that graph is *disgustingly* colored, and the legend is kind of annoyingly named. Let's fix both of those things:

```
myForeignHistogram <- ggplot(autos, aes(x = weight, fill = foreign, color = foreign)) +  
  # make the histograms transparent and not stacked  
  geom_histogram(bins = 10, alpha = 0.4, position = "identity") +  
  ggtitle("Vehicle Weight Histogram (by Origin)") +  
  xlab("Weight (lbs)") + ylab("Number of Cars") +  
  myThemeStuff +  
  # set up the fill for the 2 histograms  
  scale_fill_manual(values = c("NA", "blue")) +  
  # and the color  
  scale_color_manual(values = c("black", "darkblue")) +  
  theme(legend.title = element_blank())
```

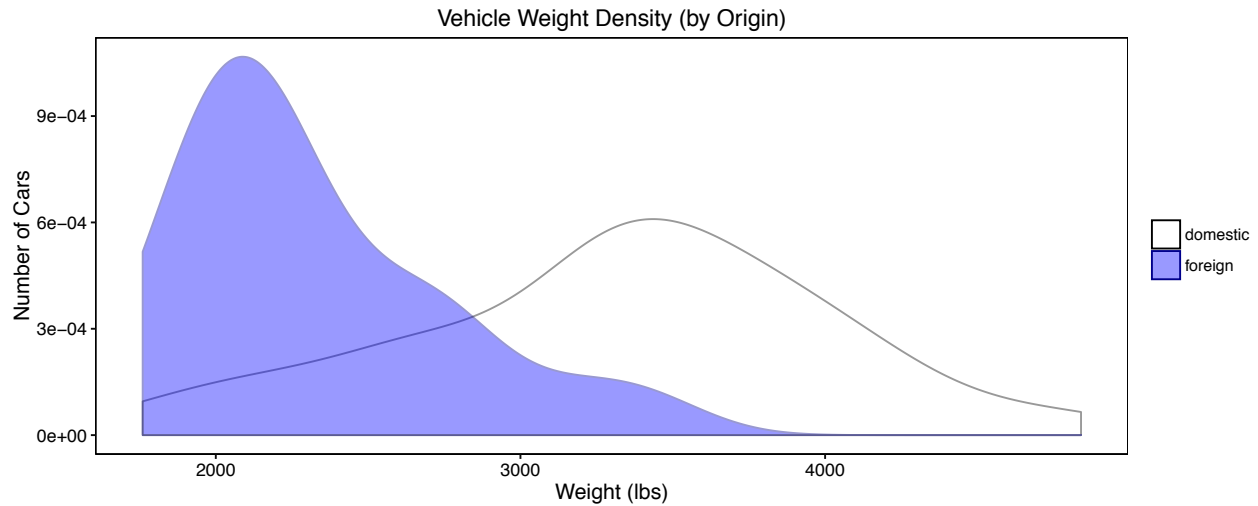
myForeignHistogram



We can also turn this into a nice kernel density:

```
myForeignDensity <- ggplot(autos, aes(x = weight, fill = foreign, color = foreign)) +  
  # make the densities transparent. not stacked is the default  
  geom_density(alpha = 0.4) +  
  ggtitle("Vehicle Weight Density (by Origin)") +  
  xlab("Weight (lbs)") + ylab("Number of Cars") +  
  myThemeStuff +  
  # set up the fill for the 2 histograms  
  scale_fill_manual(values = c("NA", "blue")) +  
  # and the color  
  scale_color_manual(values = c("black", "darkblue")) +  
  theme(legend.title = element_blank())
```

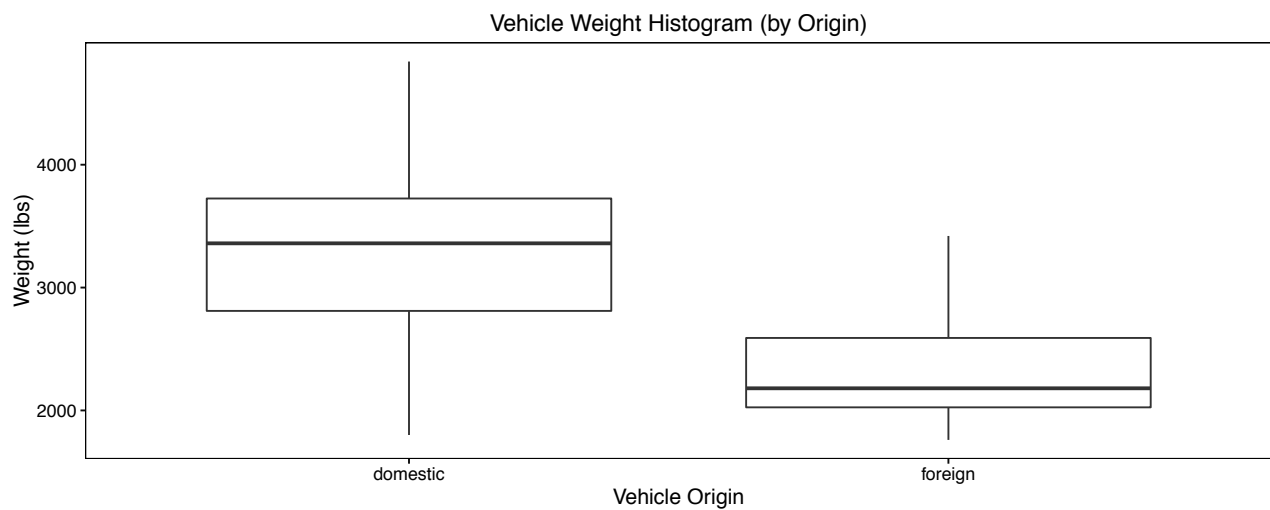
```
myForeignDensity
```



That's illuminating. What about putting a box plot together showing the same thing?

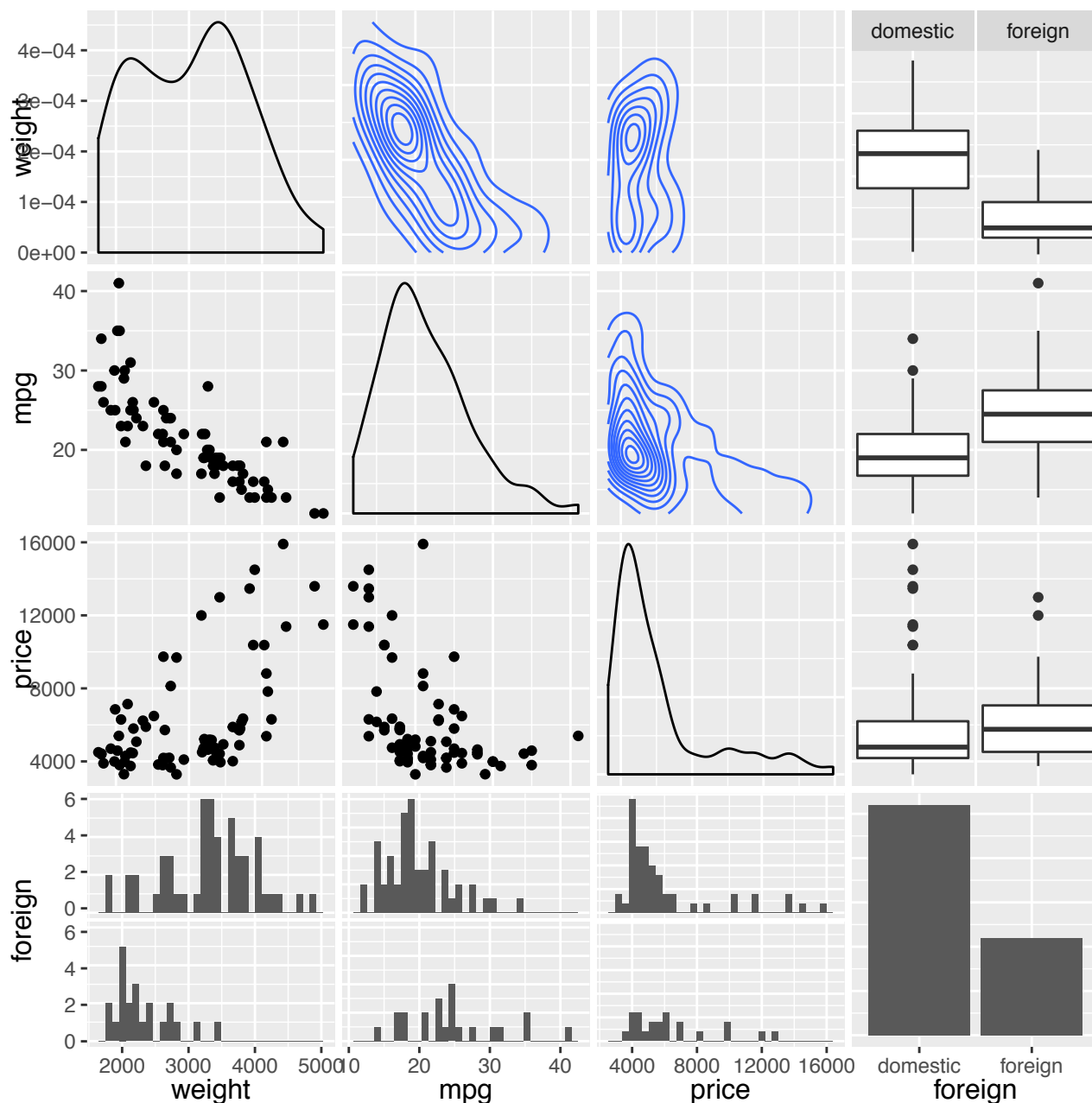
```
myForeignBoxPlot <- ggplot(autos, aes(y = weight, x = foreign)) +  
  geom_boxplot() +  
  ggtitle("Vehicle Weight Histogram (by Origin)") +  
  xlab("Vehicle Origin") + ylab("Weight (lbs)") +  
  myThemeStuff
```

```
myForeignBoxPlot
```



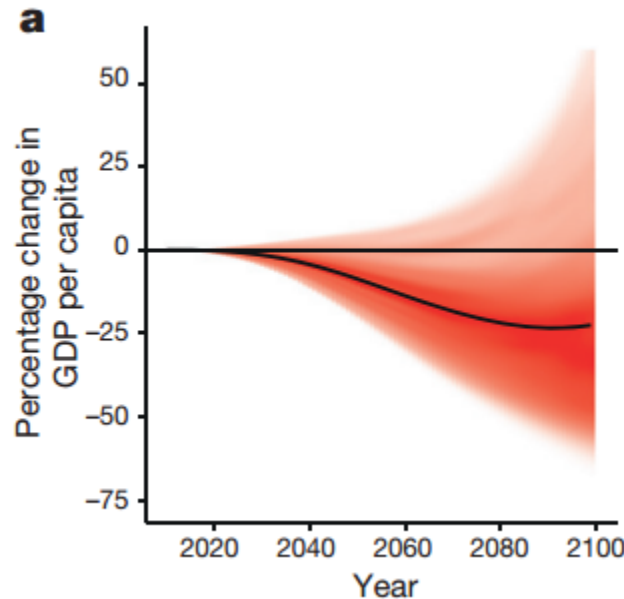
With great power comes great responsibility

Before we finish, I want to leave you with a couple of words of caution about your newfound `ggplot2` skills. You now have an incredibly powerful tool at your disposal, with nearly infinite customization options. That said, though, I strongly urge you to remember that as econometricians, your figures should first be about displaying data in a way that is informative and persuasive, and second about being nice aesthetically. It can be tempting to push `ggplot2` to the extreme because you can, to make things like this:



I'm not even giving you the code to do this, because it is an unhelpful way of displaying your data (and also looks like crap). Be thoughtful and careful in the way you put your figures together, rather than letting R slap plots together for you.

That was one extreme - it both looked bad and was uninformative. I'm also going to show you an example from a Real Published Paper where I (personally) think that the authors let function follow form.¹² First, let me just say that I think this paper is super interesting, and most of the figures this research team makes are great. In fact, here's one example of a figure from the same paper that does a really good job of getting its point across (it was even made in R):



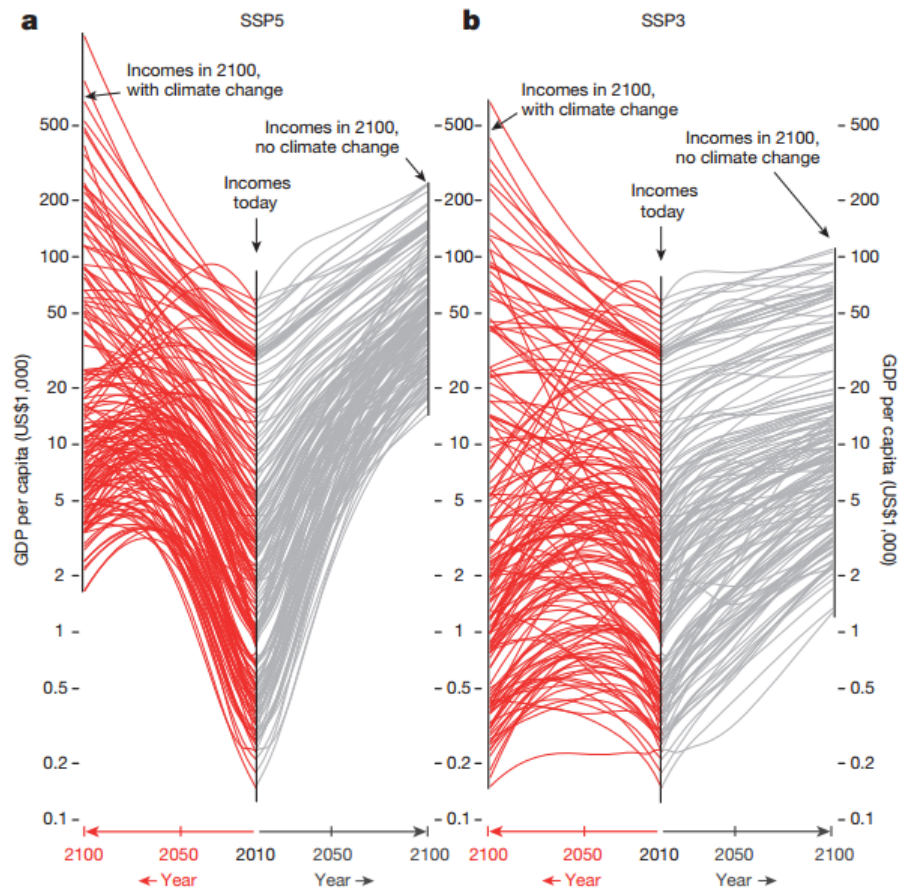
This figure is incredibly clear: the axes are nicely labeled; the actual regression line stands out in black, and the bootstrapped confidence interval is shown in red, where a darker color indicates that more bootstrap iterations pass through a given point. The zero line (the null hypothesis) is also clearly shown. The message of this figure is obvious: By 2040, global GDP per capita will fall by approximately 25% - with large uncertainty - as a result of climate change.¹³ Holy s*\$@!

On the other hand, here's a second figure from the paper. I think this figure is beautiful - I could print it out and hang it on my wall - but I personally think it's hard to figure out what's going on here without extensive notes below (which I haven't reproduced here). Feel free to form your own opinion about this (and every other) figure - I'm just giving you my two cents.¹⁴

¹²Figures reproduced without permission from a Fancy Scientific Journal.

¹³Okay, this figure by itself doesn't say anything about climate change - but that's actually obvious from the paper.

¹⁴Don't tell the authors; one of them might end up on my dissertation committee!



ggplot() holes

Still not quite satisfied with what we've done in class today? Good - you shouldn't be! We've only scratched the surface of what `ggplot2` can do. We'll keep using it over the semester, but I urge you to check it out on your own too. There are a ton of resources out there that detail `ggplot2`'s various options. A few of my favorites are:

- The `RStudio` data visualization cheat sheet, found on [this page](#), with a bunch of other useful resources.
- Zev Ross' [blog post](#) on "Beautiful Plotting in R".
- Another [useful blog post / tutorial](#) from Ramnath Vadiyanathan.
- Part of a statistics course with a [thorough introduction](#) to `ggplot2`,
- The official `ggplot2` [documentation](#) is helpful, if a little opaque.